Networks & Distributed Programming

# The Application Layer: HTTP Messages and Caching

## Lecture 5



UNIVERSITÀ DEGLI STUDI DI NAPOLI
FEDERICO II



DIETI

UNINA

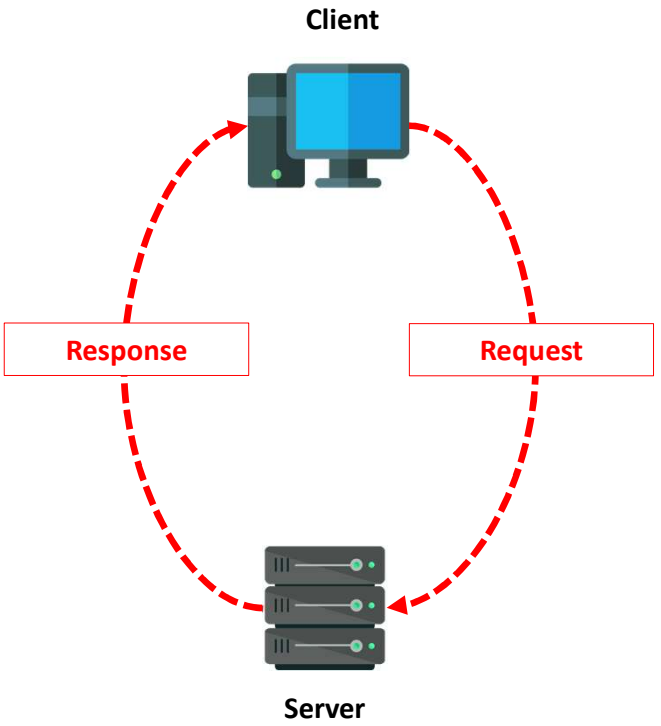
# Application Layer
## HTTP recap

- **HyperText Transfer Protocol** (HTTP):
  - It is one of the most used protocol on internet (**web-based** applications).
  - It is a **client-server** protocol, HTTP defines:
    - the **structure of messages** (message format).
    - how **client and server exchange** such messages.
      - NOTE: how client and server applications should be implemented is not decided by HTTP.
  - It uses **request/response** types of messages.
  - It is **stateless**: client and server do not remember past interactions.
  - It assumes **reliable** and connection-oriented transport protocol (mostly TCP and recently UDP).
    - It may use **persistent** and **non-persistent** connections.

# Application Layer

## HTTP: Message Format

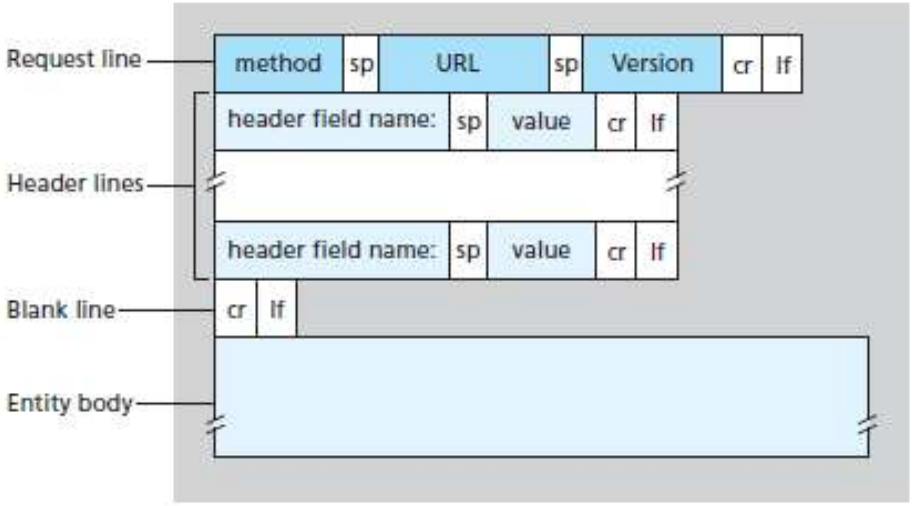
- In HTTP protocol we have 2 different formats for the request and the response messages.
- **Request**: specifies the **command** (method) that the HTTP server has to perform (e.g., give me a web page, fill a specific form, etc.).
- **Response**: reports the **outcome** of the command (success, fail, etc.) and possible data (e.g., the requested file).
- Both messages are written in ordinary **ASCII text**, so they are easily readable by humans.



# Application Layer

## HTTP: Request Message Format

- The request message format include the following fields:
  - The **method**: specifies the requested command to be executed by the server.
  - The **URL**: is used to identify the object on which we want to operate.
  - The **version**: specifies the HTTP version (e.g., HTTP/1.1).
  - The **header lines**: contain the parameters of the request, the number and the type of these lines are not fixed. Each line include the **name** and the **value** of the parameter.
    - For instance, we can here specify if we want **persistent** or **non-persistent** connection.
    - Custom headers can also be used.
  - The **body**: is method-specific and contains data that are potentially associated with the command (e.g., text used to fill a form).




The fields are separated by special characters:

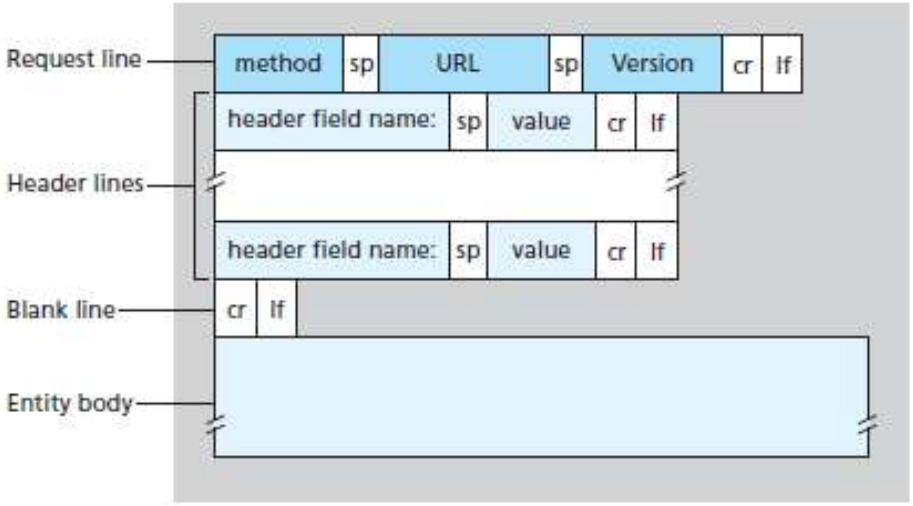
- The sp is space character.
- The cr is carriage return (\r).
- The lf is line feed (\n).

# Application Layer

## HTTP: Request Message Format (Methods)

- The **GET** method is used to retrieve objects (resources) form the server.
  - This is one of the most used commands as every time we browse a new webpage the associated file must be retrieved.
  - In the GET method the **body is empty**.

- The **POST** method is used to set info inside objects (resources) of the server.
  - A request generated with a form does not necessarily use the POST method, HTML forms often use the GET method and include the data (in the form fields) in the requested URL.
  - The body contains the info to be posted.



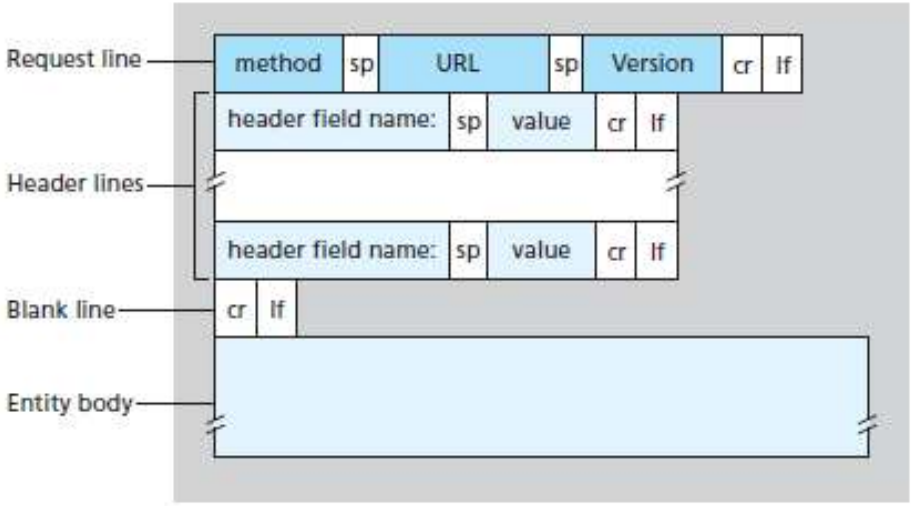
# Application Layer

## HTTP: Request Message Format (Methods)

- The **HEAD** method is similar to the GET method, when a server receives a request with the HEAD method, it responds with an HTTP message, but the object is not returned.
  - Application developers often use the HEAD method for debugging.

- The **PUT** method is often used in conjunction with Web publishing tools.
  - It allows a user to upload an object to a specific path (directory) on a specific Web server.
  - The PUT method is also used by applications that need to upload objects to Web servers.

- The **DELETE** method allows a user, or an application, to delete an object on a Web server.



# Application Layer

## HTTP: Request Message Format (Example)

- In this example we have a **browser** (Mozilla/5.0), implementing “HTTP/1.1”, that is **requesting** the object “/somedir/page.html” to the “www.someschool.edu” web server.

```
GET /somedir/page.html HTTP/1.1
Host: www.someschool.edu
Connection: close
User-agent: Mozilla/5.0
Accept-language: fr
```

- We have **five lines**:
  - The **GET request line** (resource and version).
  - The body is not used (empty).
  - The **four header lines** (host, connection, user-agent, accept-language)…

# Application Layer

## HTTP: Request Message Format (Example)

- The line **“Host: www.someschool.edu”** specifies the **host on which the object resides**.
  - **Destination is not always the next host**: a message could be **forwarded by another host** (e.g., proxy server), so the address in the TCP could be different from the real target host.
- The line **“Connection: close”** specifies that the browser is asking for a **non-persistent connection** (i.e., close after finish).
- The line **“User-agent: Mozilla/5.0”** specifies the **browser’s type**.
  - This is sometimes useful because the **server can send different versions** of the same object (each version is addressed by the same URL) **depending on the type**.
- The line **“Accept-language: fr”** indicates that the **user prefers to receive a French version** of the object (if exists).

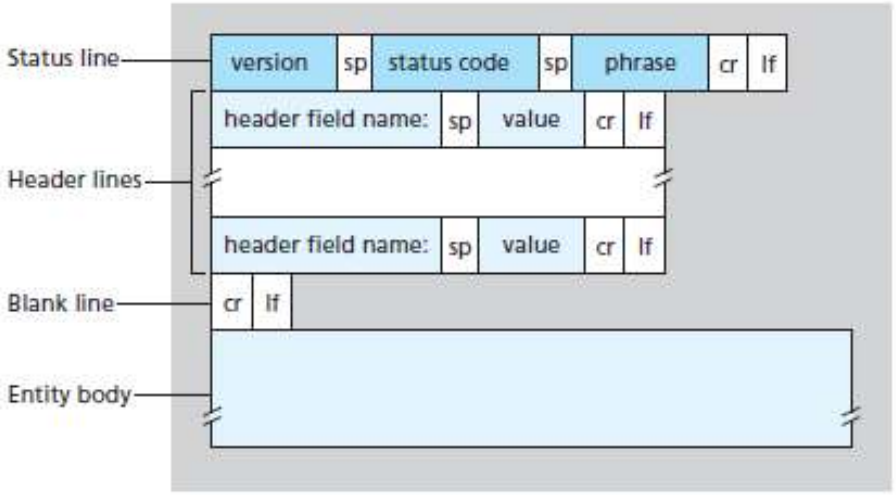
```
GET /somedir/page.html HTTP/1.1
Host: www.someschool.edu
Connection: close
User-agent: Mozilla/5.0
Accept-language: fr
```

# Application Layer

## HTTP: Response Message Format

- The general format of the response message is similar to the request one.

- In this case, instead of a request line, we have a **status line** that reports the outcome of the command that includes:
  - The **version**: reports the HTTP version of the server's response.
  - The **status code**: a code (number) that specifies the outcome of the command.
  - The **phrase**: contains the result of the request.



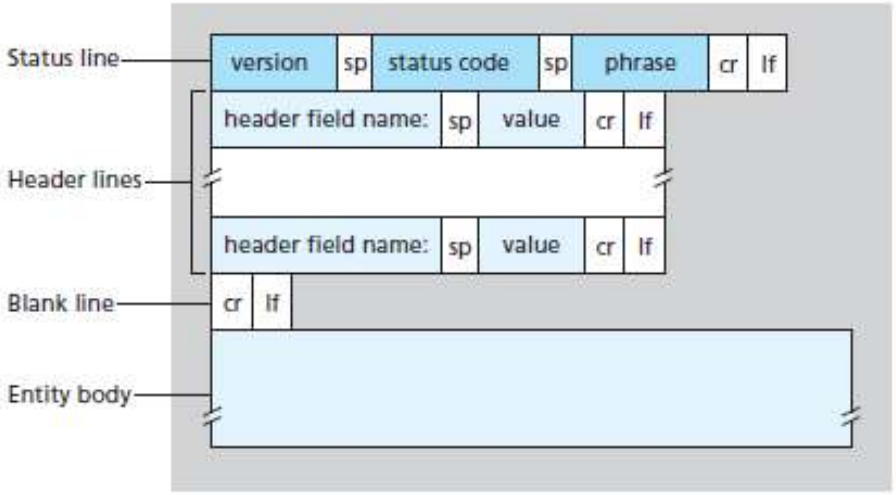

HTTP/1.1 200 OK
- HTTP/1.1 = version
- 200 = status code
- OK = phrase

HTTP/1.1 404 Not Found
- HTTP/1.1 = version
- 404 = status code
- Not Found = phrase

# Application Layer

## HTTP: Response Message Format (Status Codes)

- **Status codes** are divided into classes:
  - 100-199 **informational**: info regarding the request.
  - 200-299 **successful**: the request has been executed successfully.
  - 300-399 **redirection**: there are additional actions needed from the client.
  - 400-499 **client-error**: the request cannot be executed because of a client issue.
  - 500-599 **server-error**: the request cannot be executed because of a server issue.

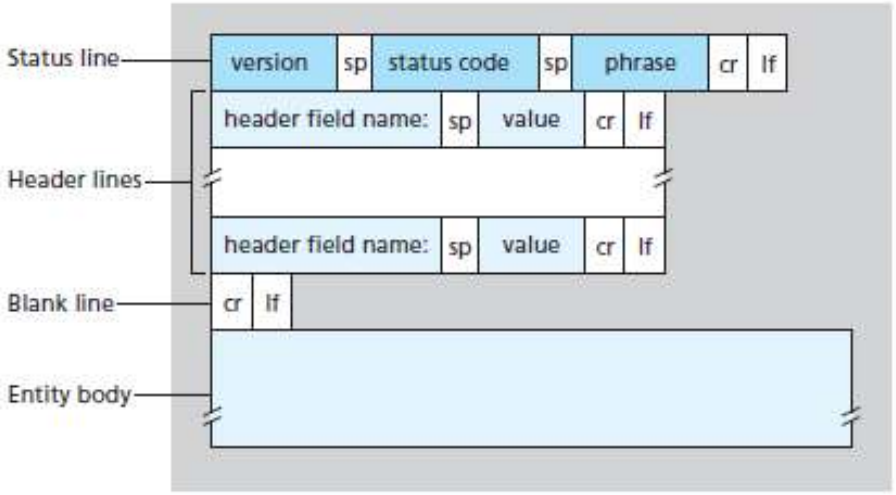



The first digit is the whole story …

# Application Layer

## HTTP: Response Message Format (Status Codes)

- Some typical examples are:
  - **200 OK**: request succeeded, and the information is returned in the response.
  - **301 Moved Permanently**: requested object has been permanently moved; the new URL is specified in the "Location" header of the response message.
  - **400 Bad Request**: generic error code indicating that the request could not be understood by the server.
  - **404 Not Found**: The requested document does not exist on this server.
  - **505 HTTP Version Not Supported**: The requested HTTP protocol version is not supported by the server.




304 Not Modified, exists only to make caching work: we come back to it later today.

# Application Layer

## HTTP: Response Message Format (Example)

- In this example, the status line indicates that **the server is using HTTP/1.1 and that everything is OK** (the server has found and is sending the requested object).

- The **body contains the requested object** (represented by data data data data data ...).

- There are **five header lines** (connection, date, server, last-modified, content-length, content-type)…

```
HTTP/1.1 200 OK
Connection: close
Date: Tue, 18 Aug 2015 15:44:04 GMT
Server: Apache/2.2.3 (CentOS)
Last-Modified: Tue, 18 Aug 2015 15:11:03 GMT
Content-Length: 6821
Content-Type: text/html

(data data data data data ...)
```

# Application Layer

## HTTP: Response Message Format (Example)

- **"Connection: close"** informs the client that the **connection will be closed** after this message (non-persistent).
- **"Date: …"** indicates the time and **date when the HTTP response was created** and sent by the server (not when the object was written).
- **"Server: …"** indicates that the message was **generated by an Apache Web server** (similar to user-agent).
- **"Last-Modified: …"** indicates the time and date when the **object was created or last modified**.
  - Useful in case of caching, as cached files can be outdated.
- **"Content-Length: …"** indicates the **number of bytes in the object**.
- **"Content-Type: …"** indicates the type (here the object is HTML text)
  - The type is settled here, not by the file extension.

```
HTTP/1.1 200 OK
Connection: close
Date: Tue, 18 Aug 2015 15:44:04 GMT
Server: Apache/2.2.3 (CentOS)
Last-Modified: Tue, 18 Aug 2015 15:11:03 GMT
Content-Length: 6821
Content-Type: text/html

(data data data data data ...)
```

# Application Layer

## HTTP: Talking HTTP by hand

- Typing a HTTP request using netcat (nc)
  - -C is not optional: HTTP ends every line with CRLF, and nc alone sends only LF. Try it, and this server answers 400 Bad Request.
  - Use nc to connect to the first website in history, created by Tim Berners-Lee

```
nc -C info.cern.ch 80          opens a TCP connection to port 80
GET /hypertext/WWW/TheProject.html HTTP/1.1
Host: info.cern.ch
                               the blank line ends the request
HTTP
```

  - Replace GET with HEAD and you get the headers alone — the same metadata, none of the traffic.

# Application Layer

## HTTP: Statelessness and Cookies

- An HTTP **server is typically stateless**, this simplifies server design reduces the use of resources and allow servers to handle thousands of simultaneous TCP connections.

- Pure statelessness is also a strong limitation as **several web functions are client-specific** (shopping carts contains several requests, recommendations, preferences during sessions, etc.).

- The identity has to be carried in the messages, since it cannot live in the server's memory.

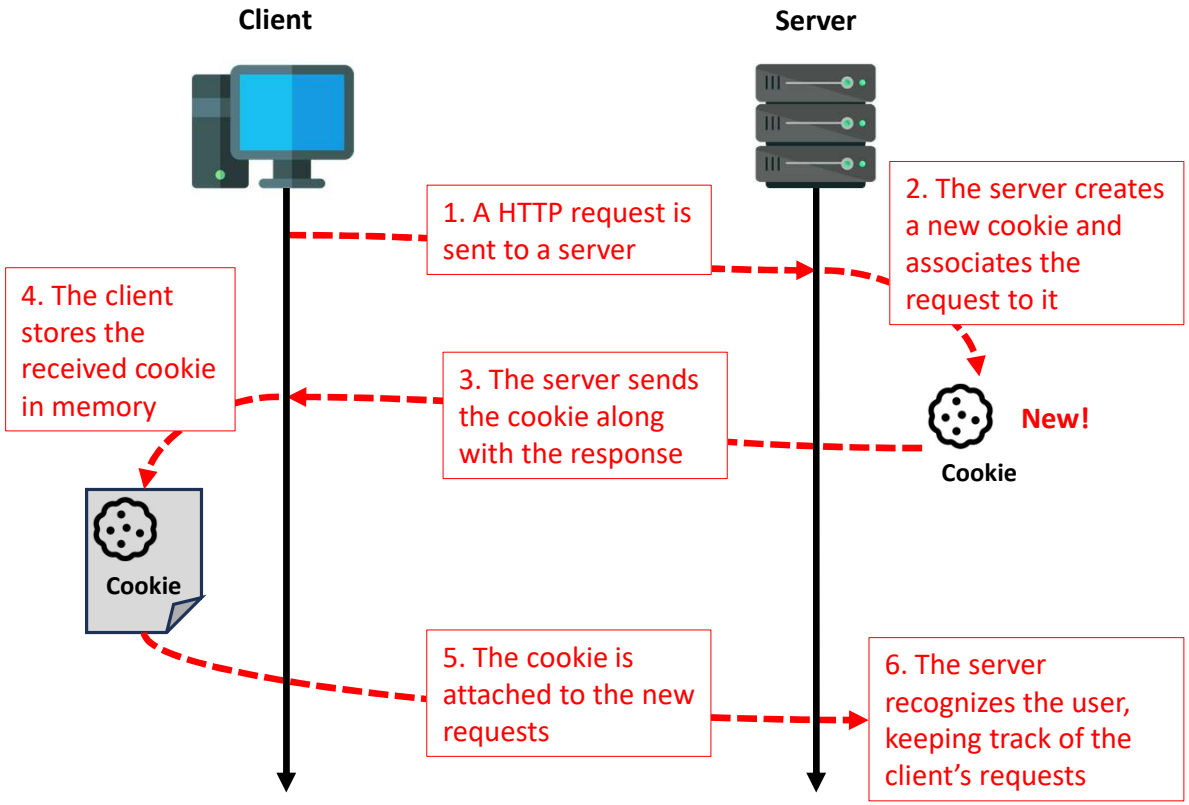
- For these purposes, HTTP uses **cookies**.
- A cookie is a digital token used by servers to identify clients.
  - Cookies allow web sites to keep track of users.
  - **Alphanumeric ID** that a server hands to a client, and that the client hands back on every later request

# Application Layer
## HTTP: Cookies

- A **cookie is created by the server** and delivered to the client.

- Cookie technology involves four main components:
  1. A **"Set-cookie" header** line in the HTTP response message.
  2. A **"Cookie" header** line in the HTTP request message.
  3. A **file** on the client system (managed by the user's browser).
  4. A **back-end database** on the server, indexed by that id

Note where the state actually is: at the two ends, never in the protocol.

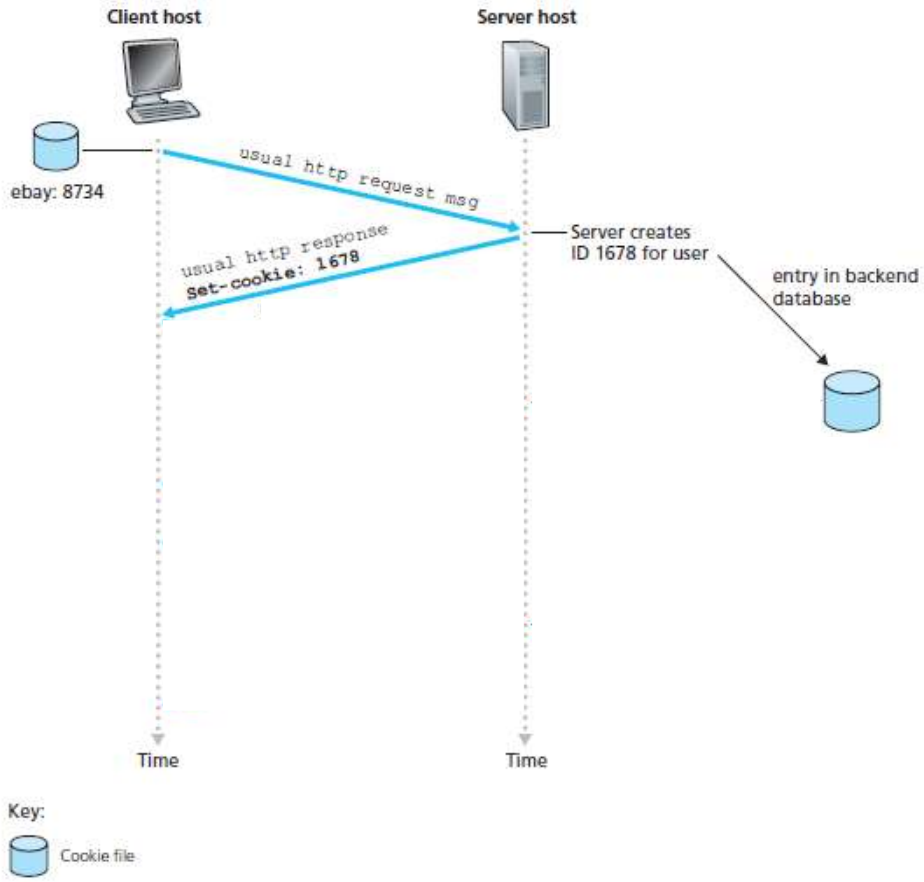



The cookie is created by the server, stored by the client, and returned by the client from then on.

# Application Layer

## HTTP: Cookies (Example)

- Let's assume a client host, which uses eBay regularly, to **contact Amazon.com for the first time**.
- The client already has a cookie for eBay, but **it has no cookie for Amazon**.
- When the request comes into the Amazon Web server, **the server creates a cookie** (ID number) and an associated entry in its back-end database.
- The Amazon Web server then responds to client's browser and **includes in the HTTP response a "Set-cookie" header** line, which contains the ID (`Set-cookie: 1678`).



# Application Layer

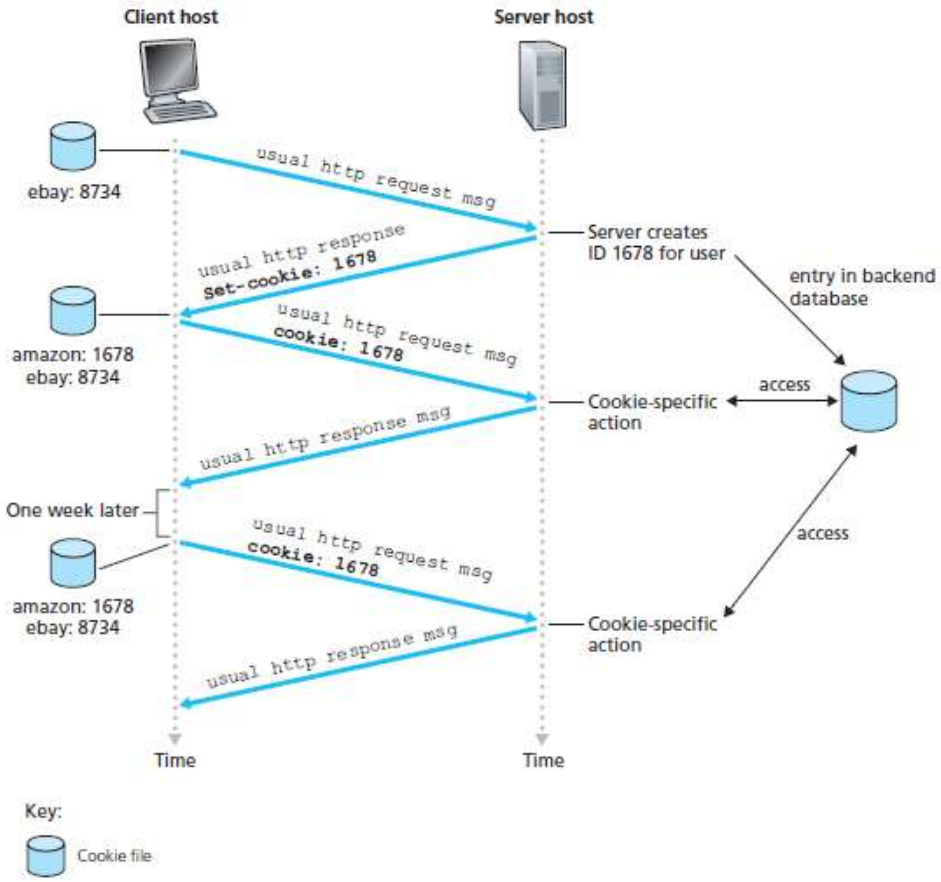
## HTTP: Cookies (Example)

- When the response is received, **the browser appends a line to a special cookie file** including:
  - the hostname of the server.
  - the ID number of the cookie.

- From now on, the requests from the client to Amazon **will be associated to the new cookie** by including the header line `Cookie: 1678` to the HTTP messages.

- Amazon server is then able to **track the client's activity through the database**. It knows exactly which pages user 1678 visited, in which order, and at what times.

- If the client returns to Amazon's site **one week later**, the browser will continue to put the header line Cookie: 1678 in the request messages.



# Application Layer
## HTTP: Cookies

- You clear your browser data mid-purchase. What happens to the cart?

# Application Layer
## HTTP: Cookies

- You clear your browser data mid-purchase. What happens to the cart?
  - The next request arrives without the Cookie: line. The server has no way to tell it from a first visit, so it issues a new identifier: the cart is empty, and the old one is still in the database, now unreachable.
    - The server was never keeping your session: it was keeping a row, and you were carrying the key to it. Which is also why the same account looks different on your phone: a different file, a different key.

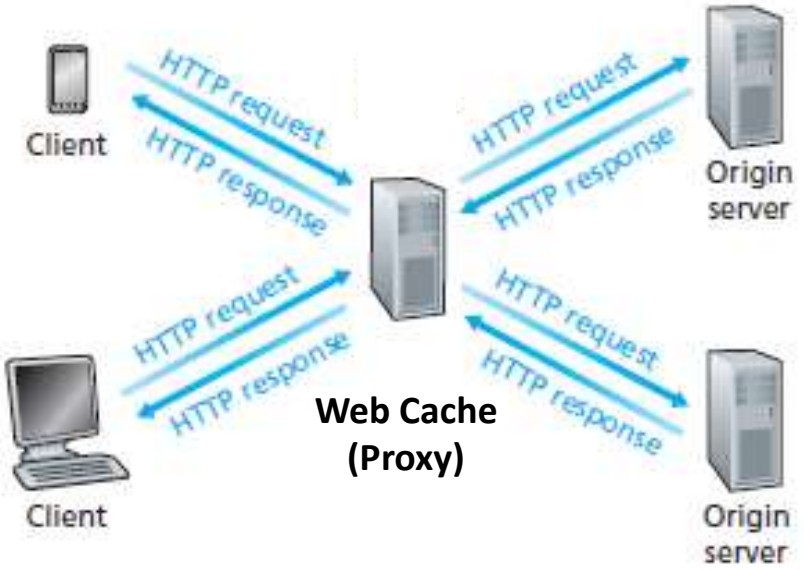
HTTP has not become stateful.

The state was built on top of it, out of two header lines.

# Application Layer

## HTTP: Web Caching

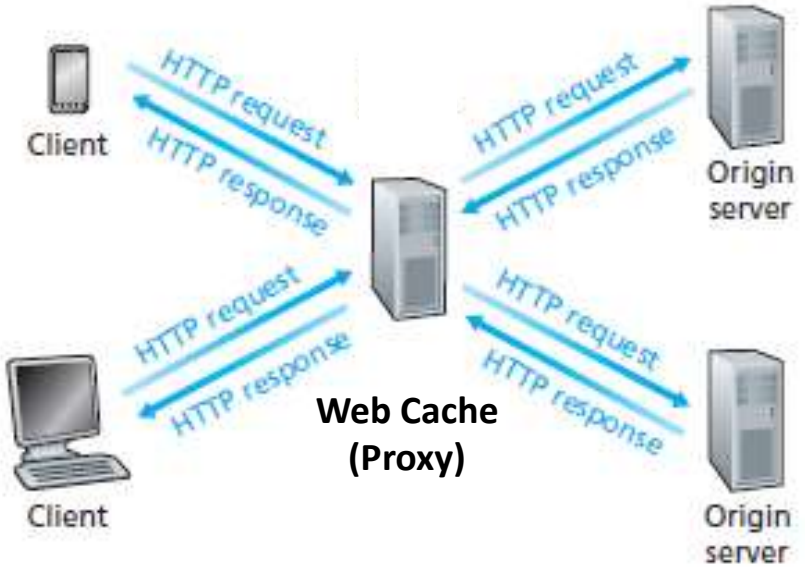
- A **Web cache** is a network entity that satisfies HTTP requests on the behalf of an origin Web server.
  - This kind of middle entities working on behalf of others are called **proxy servers**.
- The Web cache has its own disk storage and **keeps copies of recently requested objects** in this storage.
- A browser can be configured so that all the HTTP **requests are first directed to the Web cache** to check if a copy of the requested object is available.



# Application Layer

## HTTP: Web Caching

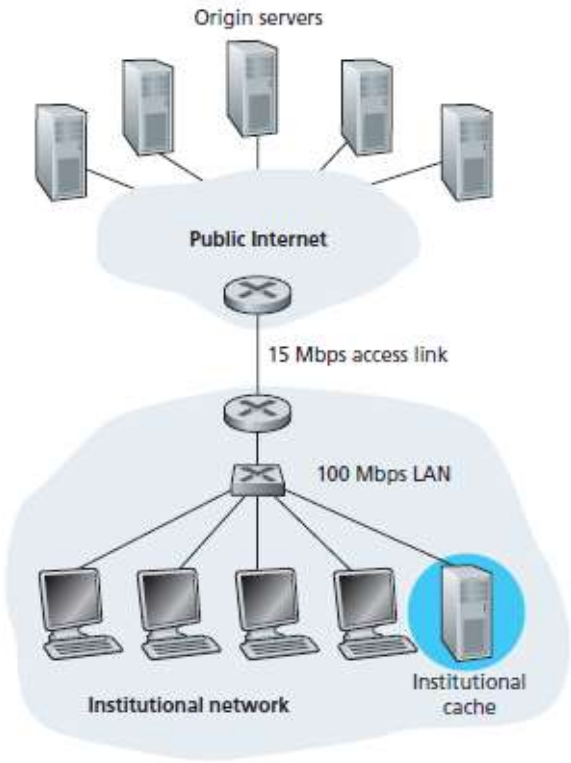
- Let's assume a browser to request the object http://www.someschool.edu/campus.gif passing through a web cache:
  1. The **browser establishes a TCP connection** to the Web cache and sends an HTTP request for the object to the Web cache.
  2. The Web cache **checks if it has a copy of the object** stored locally. If so, the Web cache returns the object within an HTTP response message to the client browser.
  3. If the **Web cache does not have the object**, the Web cache opens a TCP connection to the origin server (www.someschool.edu) and sends an HTTP request for the object.
  4. When the **Web cache receives the object, it stores a copy in its local storage** and sends a copy, within an HTTP response message, to the client browser.



# Application Layer
## HTTP: Web Caching

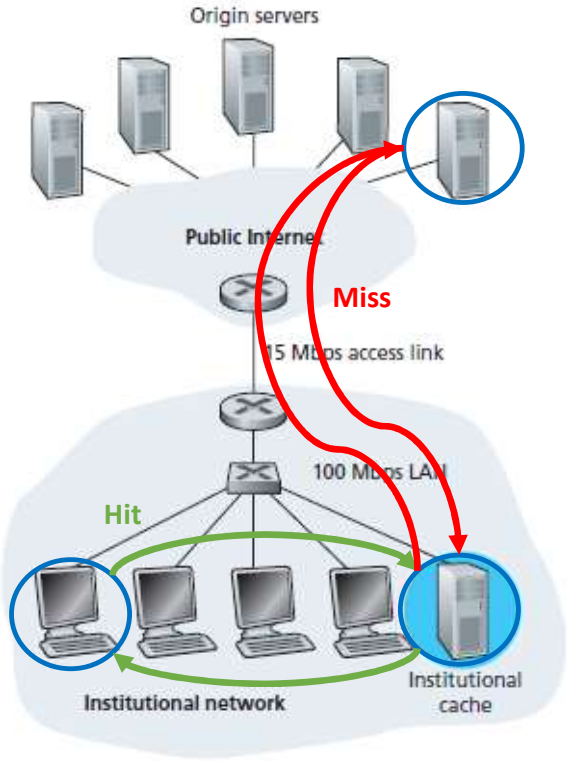
- Note that a cache is both a server (when providing objects) and a client (when requesting objects) at the same time.

- Web caching has seen deployment in Internet for two reasons:
  1. A Web cache can substantially **reduce the response time for a client request**, particularly if a high-speed connection stands between the client and the cache.
  2. Web caches can substantially **reduce traffic** of a company or institution toward Internet, in so reducing costs due to bandwidth.



# Application Layer
## HTTP: Web Caching

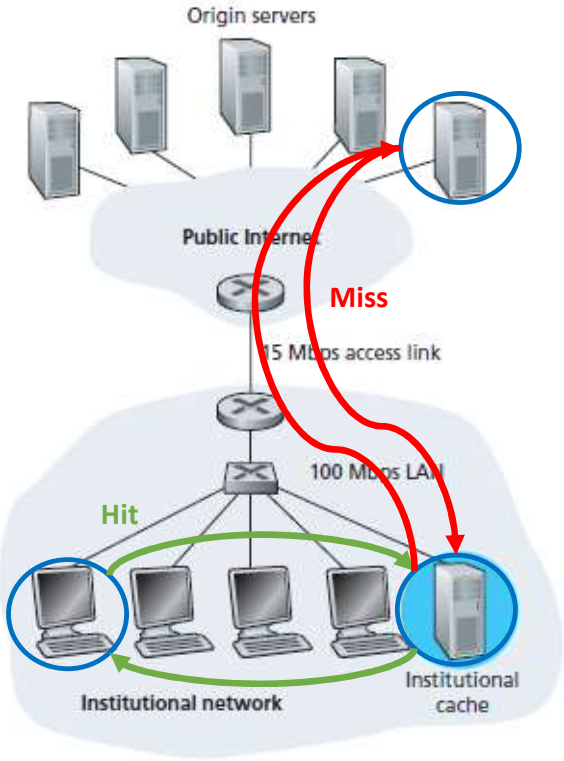
- Web caches are **often installed into a company/institution local network** to speed-up and reduce traffic.
- A **hit** happens when a cache successfully provide an object without contacting the original server.
- The hit rate, i.e., the fraction of requests that are satisfied by a cache, typically **ranges from 0.2 to 0.7**.
  - Hit rate increases when more clients use the cache.
- This means that up to 70% of requests can be served locally.



# Application Layer

## HTTP: Web Caching

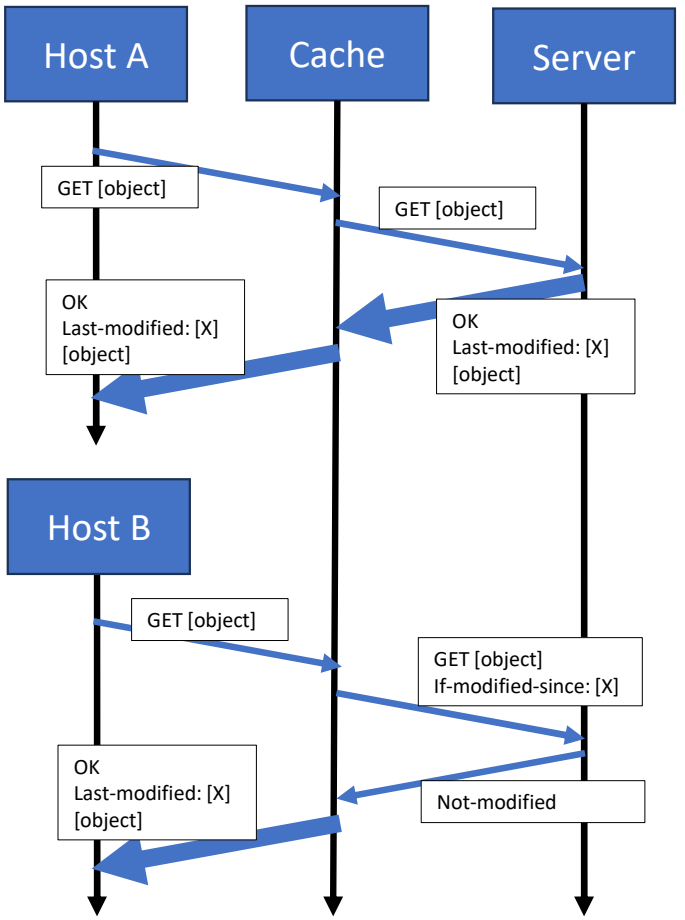
- Average object 1 Mbit, 15 requests per second from the institution, and two seconds on average to fetch anything from the far side of the Internet.
- Demand on the access link: 15 × 1 = 15 Mbps, on a link of exactly 15 Mbps.
- A link served at the whole of its capacity does not keep up: the queue in front of it grows without bound, and the wait becomes minutes.
- Upgrading the link to 100 Mbps fixes it, and costs money every month.
- A cache with hit rate 0.4 leaves only 60% of the requests to cross: demand falls to 9 Mbps, and the average wait to about 1.2 seconds (hit x 0 + miss x 2).
- Cheaper than the upgrade, and faster than it. Why the queue behaves that way is a question for the transport layer.



# Application Layer

## HTTP: Web Caching

- Web caching introduces a new problem: the copy of an object residing in the **cache may be outdated**.
- To avoid this issue, HTTP has a mechanism that allows a cache to **verify the stored object**.
- The **conditional GET** is an HTTP request message including a GET method and “If-Modified-Since:” header.
- The cache **checks if the object is up to date** and, if so, the stored version is sent back to the host (no further communication needed).
- If unchanged, the server answers 304 Not Modified with an empty body, and the cache serves its own copy.



# Application Layer

## HTTP: Web Caching

What the cache stored, one week earlier:

```
HTTP/1.1 200 OK
Last-Modified: Wed, 9 Sep 2015 09:23:24      kept with the object
```

What it sends today, and what comes back:

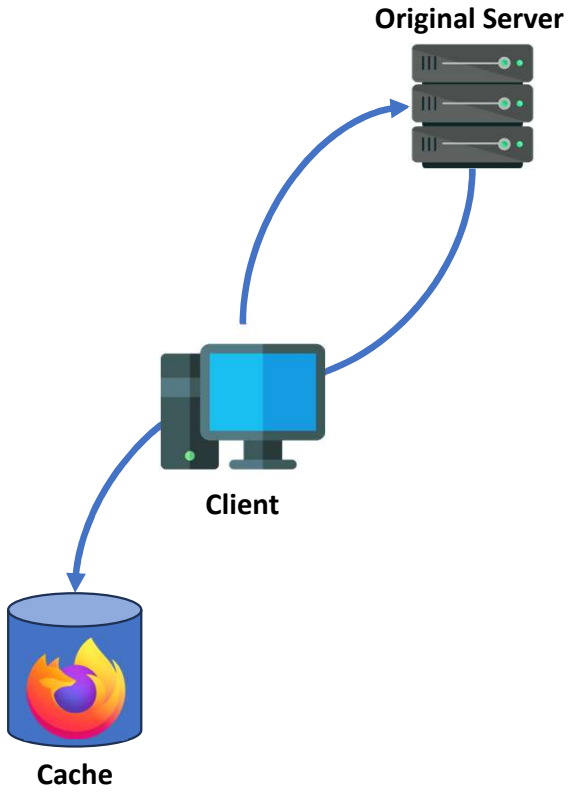
```
GET /fruit/kiwi.gif HTTP/1.1
Host: www.exotiquecuisine.com
If-modified-since: Wed, 9 Sep 2015 09:23:24 the same value, returned

HTTP/1.1 304 Not Modified
Date: Sat, 10 Oct 2015 15:39:29
                                            empty entity body
```

The image was never sent. That is the entire saving, and it is why `Last-Modified:` is on every response.

# Application Layer
## HTTP: Web Caching

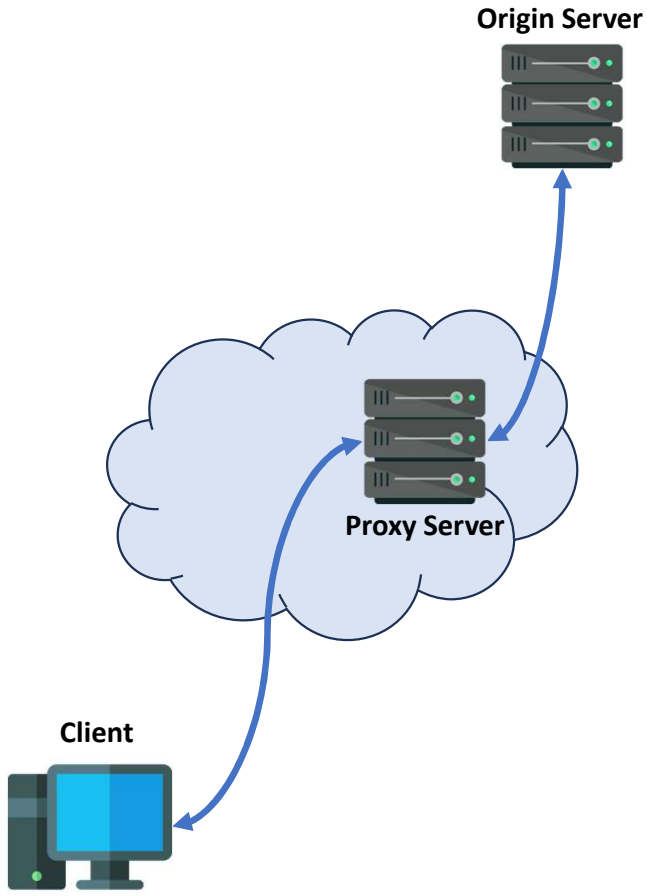
- Caching **is also performed locally** by browsers.
- The principle is the same as servers, the browser stores objects locally so they no need to be retrieved from the server.
- This is a common techniques in modern browsers as **it drastically improves performance**.
- The local version of **the object may be not updated**, generating errors (quite frequent).



# Application Layer

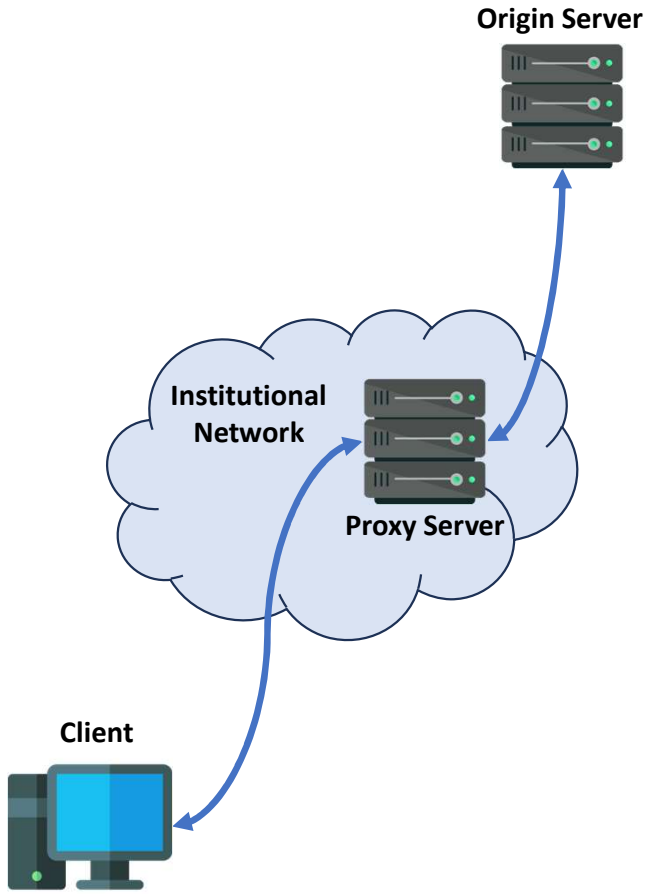
## HTTP: Proxy Servers

- A proxy server is an intermediary between a client and an origin server:
  - Client → Proxy Server → Origin Server
- The client sends requests to the proxy instead of directly to the origin server.
- The proxy forwards the requests and returns the responses to the client.
- A proxy can provide additional functions, such as **caching**, **access control**, **filtering**, or **institutional access**.



# Application Layer
## HTTP: Proxy Servers

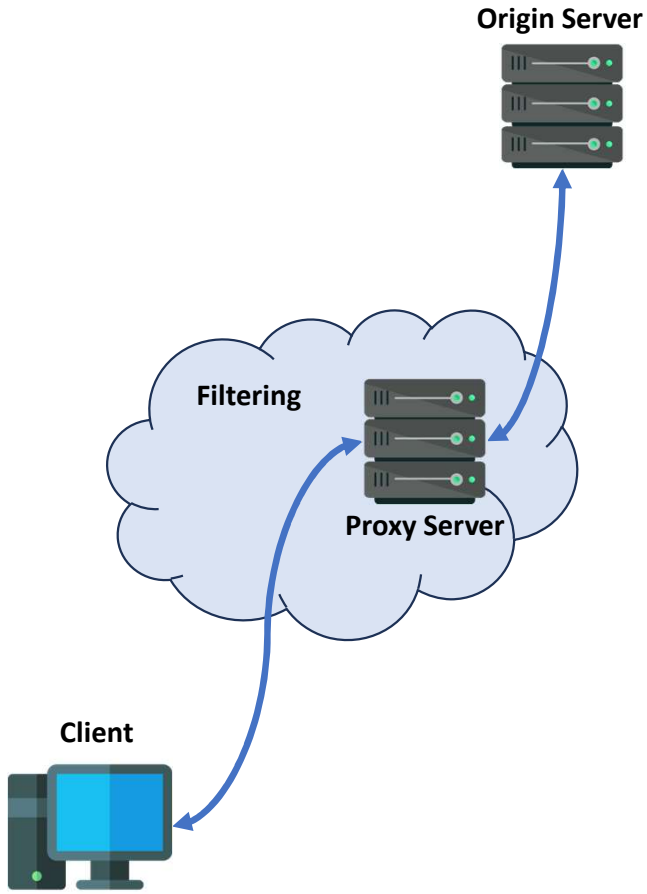
- Proxy servers can be used to access institutional services
- The UNINA network provides an **institutional** web proxy (proxy.unina.it).
- Since requests are forwarded by the institutional proxy, from the origin server's point of view they appear to come from the institution.
- There are also several commercial/free **proxy servers available on Internet**.



# Application Layer

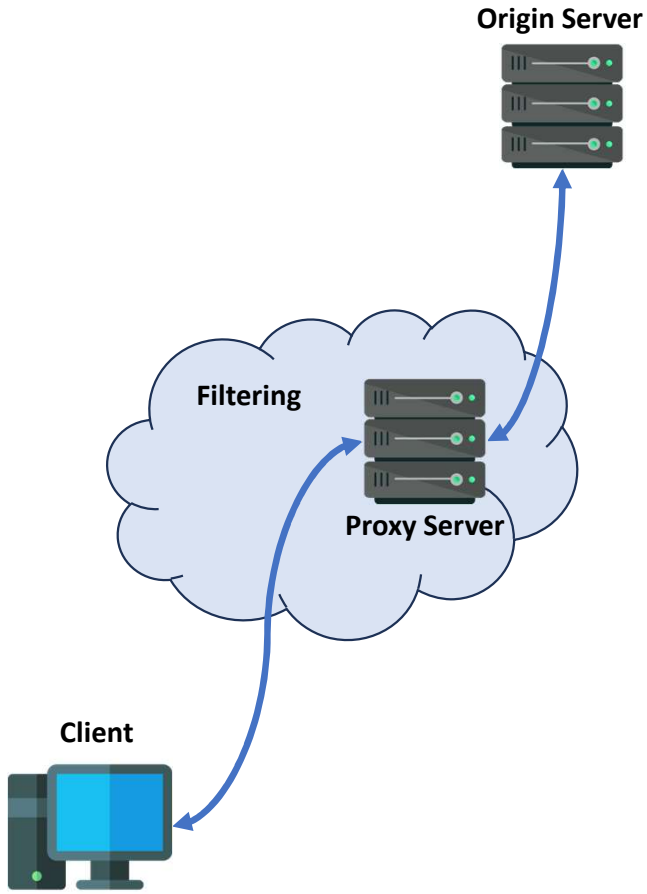
## HTTP: Proxy Servers

- Proxy servers can filter traffic using application-level information when that information is available to the proxy
- A proxy can apply access policies, for example by blocking requests to specific websites, domains, or content categories.
- A proxy can filter HTTP traffic based on application-level information, such as URLs, domains, or HTTP methods.
- With HTTPS, this information is encrypted unless the proxy performs TLS termination/interception.
- Since traffic passes through the proxy, filtering policies can be enforced centrally for many clients.



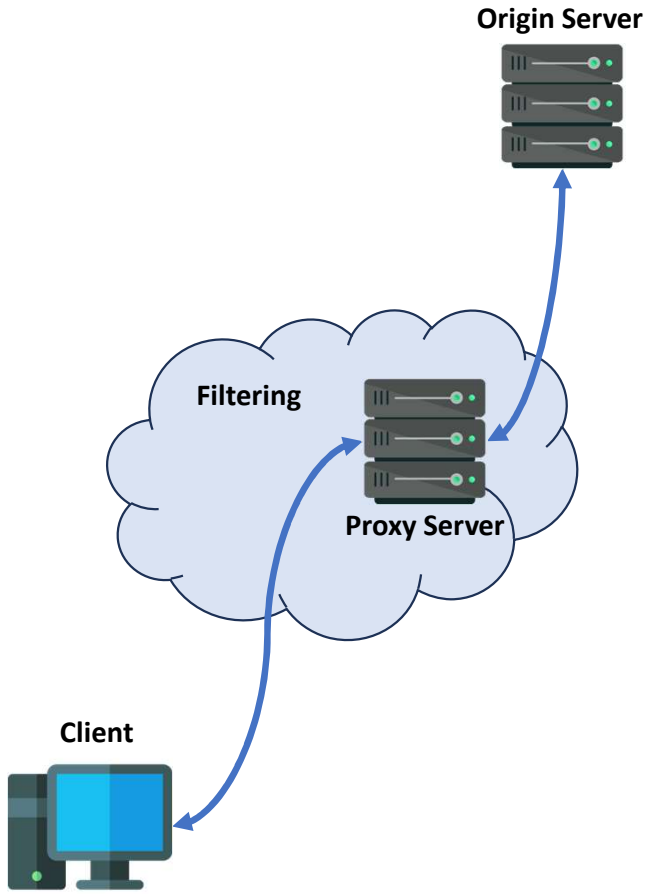
# Application Layer
## HTTP: Proxy Servers

- What about web caching with HTTPS?



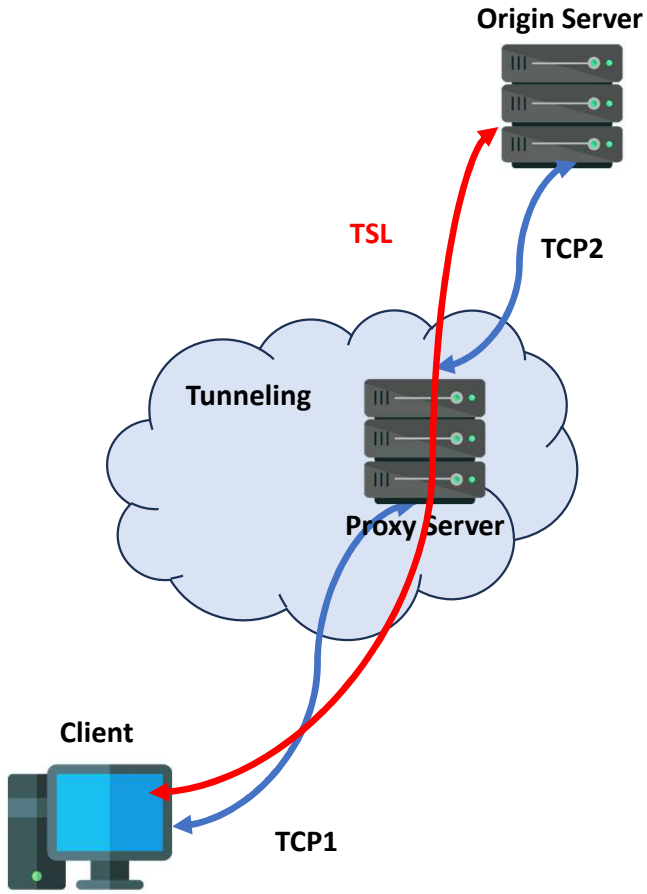
# Application Layer
## HTTP: Proxy Servers

- What about web caching with HTTPS?
  - Caching requires visibility of the HTTP messages.
  - With end-to-end HTTPS, an intermediary proxy cannot inspect or cache HTTP content.
  - HTTPS caching at the proxy requires TLS interception.
  - Client ⇄ TLS ⇄ Proxy ⇄ TLS ⇄ Origin Server



# Application Layer
## HTTP: Proxy Servers

- HTTPS tunneling
  - Client asks the proxy to open a connection to the origin server (CONNECT),
    - e.g., CONNECT www.example.com:443
  - The proxy establishes a second TCP connection to the origin server
  - The proxy then relays bytes between the two TCP connections, creating a tunnel
  - The TLS session remains end-to-end between the client and the origin server.
  - The proxy does not decrypt the HTTPS traffic.




Unlike encapsulation-based tunnels, the proxy simply relays the byte stream between two TCP connections.

# Application Layer

## HTTP: https

- Type http:// into a browser today and it is very unlikely that plain HTTP ever leaves your machine.
- HTTPS: HTTP carried over TLS, typically on **port 443**. The HTTP **semantics are unchanged**, but the HTTP messages are **encrypted in transit**.
- HSTS: a **header line** in which a site declares that browsers must never use plain HTTP for it again, for months “Strict-Transport-Security: max-age=31536000; includeSubDomains”
- Browsers ship a preload list of such sites and an HTTPS-first mode, so the URL is rewritten before any packet is sent.

So what does the http display filter show?

- Nothing. The traffic is there — TCP to port 443 — but Wireshark sees TLS records, not requests. What TLS does inside them is a question for the end of the course.

# Application Layer
## HTTP: https

- From HTTP to HTTPS
  - A client may initially request:
    http://example.com
  - The server can respond with an HTTP redirect:
    301/302 → https://example.com
  - The browser then opens a new HTTPS connection.

  - HTTPS carries HTTP messages inside TLS.
  - For HTTP/1.1 and HTTP/2:
    HTTP → TLS → TCP → IP
  - HTTPS typically uses TCP port 443.
  - HTTP contents are encrypted in transit.

# Application Layer

## HTTP: long pages

A page long enough that its response must be split across several TCP segments.

1. Empty the browser cache, so that nothing is served locally: in Firefox or Chrome, Ctrl-Shift-Del, or disable it in the developer tools.
2. Start the Wireshark capture, with no display filter yet.
3. Fetch the page with a client that will not upgrade the request:

```
$ curl http://info.cern.ch/hypertext/WWW/TheProject.html > /dev/null
```

4. Stop the capture, then type http in the display filter to find the two HTTP messages.
5. Remove the filter again: the response you are looking for is mostly in packets that are not HTTP messages at all.

# Application Layer

## HTTP: questions

A page long enough that its response must be split across several TCP segments.

(a) How many HTTP GET messages did your client send, and which packet number carries the one for the page?

(b) Which packet number carries the status code and phrase of the response?

(c) What are that status code and phrase?

(d) How many TCP segments carrying data were needed for that single response? Look for TCP segment of a reassembled PDU in the Info column.

Then repeat the whole thing with a browser instead of curl, and count the HTTP messages again.
Where did the page go?

# Application Layer
## HTTP: Summary

- An HTTP message is text: a first line, header lines, a blank line, and an optional body.
- The first line carries a method on the way out and a status code on the way back.
- Cookies build a session on top of a stateless protocol, out of two header lines and two stores.
- A cache answers on behalf of the origin server, and pays for itself on the access link rather than on the clock.
- The conditional GET keeps a cached copy honest with two small messages instead of the object.